

安全漏洞审计报告 · 第2轮

用户信息管理平台

SQL 注入漏洞审计报告

User Management Platform — SQL Injection Assessment

目标系统：用户信息管理平台 (Flask + SQLite)

报告日期：2026-07-08

漏洞等级：严重 × 2

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 新增功能与技术架构
3. 漏洞详情 — 2 项 SQL 注入漏洞
4. 攻击演示（含实况截图）
5. 修复方案
6. 修复前后代码对照
7. 修复验证
8. 修复总结

1. 报告概述

1.1 本轮背景

在第一轮密码安全修复后，系统新增了用户注册和用户搜索功能，并使用 SQLite 作为数据库。新增功能代码中故意使用 **f-string** 字符串拼接构造 **SQL** 语句，由此引入了 SQL 注入漏洞。本报告针对这两项新增漏洞进行专项审计。

1.2 审计范围

- 审计目标：注册与搜索功能的 SQL 注入风险
- 审计方法：白盒代码审计 + 黑盒注入测试
- 涉及文件：app.py（新增 register、search 路由）

1.3 时间线

2026-07-07

第一轮审计完成：8 项密码安全漏洞已修复

2026-07-08

新增注册和搜索功能，使用 SQLite 数据库

2026-07-08

第二轮审计发现：2 项严重 SQL 注入漏洞

2026-07-08

修复完成：改用参数化查询，验证通过

2. 新增功能与技术架构

2.1 新增功能

路由	方法	功能	数据库操作
/register	GET / POST	用户注册（用户名、密码、邮箱、手机号）	INSERT INTO users
/search	GET	按用户名或邮箱搜索用户	SELECT ... WHERE username LIKE ... OR email LIKE ...

2.2 数据库表结构

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  password TEXT NOT NULL,  
  email TEXT,  
  phone TEXT  
);
```

3. 漏洞详情

本轮审计发现 2 项严重 SQL 注入漏洞，均源于使用 f-string 拼接用户输入构造 SQL。

2

● 严重漏洞

2/2

✓ 已修复

VULN-09：搜索功能 SQL 注入

● 严重

位置：app.py — search 路由，第 146 行

漏洞代码：

```
def search():
    keyword = request.args.get("keyword", "")
    sql = f"SELECT ... WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
    c.execute(sql)
```

描述：keyword 参数未经任何过滤直接拼入 SQL 的 LIKE 子句。攻击者可通过闭合单引号注入任意 SQL 命令。

攻击向量：

- ' OR '1'='1 — 绕过查询条件，获取全部用户
- ' UNION SELECT ... FROM sqlite_master-- — 读取数据库结构
- ' UNION SELECT ... FROM users-- — 窃取所有用户密码

VULN-10 : 注册功能 SQL 注入

● 严重

位置 : app.py — register 路由 , 第 124 行

漏洞代码 :

```
def register():
    username = request.form.get("username")
    password = request.form.get("password")
    email = request.form.get("email")
    phone = request.form.get("phone")
    sql = f"INSERT INTO users (...) VALUES ('{username}', '{password}', '{email}', '{phone}')"
    c.execute(sql)
```

描述 : 四个表单字段全部直接拼入 INSERT 语句。攻击者可在任意字段中注入 SQL , 包括执行多条语句或插入恶意数据。

攻击向量 : 在任意输入字段中包含单引号和 SQL 命令 , 即可篡改 SQL 语义。

4. 攻击演示

4.1 搜索注入 — 绕过查询获取所有用户

```
// 攻击请求: keyword = ' OR '1'='1'
GET /search?keyword=' OR '1'='1'

// 实际执行的 SQL (从服务器控制台日志截取)
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'
↑ 永真条件 '1'='1' 使 WHERE 始终成立, 返回全部用户
```

结果: 原本只想搜索某个用户, 注入后返回了数据库中所有用户的记录。

4.2 搜索注入 — UNION 查询窃取表结构

```
// 攻击请求: keyword = ' UNION SELECT name,sql,2,3 FROM sqlite_master--
GET /search?keyword=' UNION SELECT name,sql,2,3 FROM sqlite_master--

// 实际执行的 SQL
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' UNION SELECT name,sql,2,3 FROM sqlite_master--%'
↑ UNION 注入 + 注释符, 读取系统表 sqlite_master
```

结果: 成功读取到完整的建表语句: CREATE TABLE users (id INTEGER ..., username TEXT ..., password TEXT ..., ...). 攻击者由此获知数据库完整结构, 为后续攻击铺路。

4.3 注册注入 — 注入恶意数据

```
// 攻击请求: username = 'hacker', 'hacked', 'hack@hack.com', '000'); --
POST /register username="hacker', 'hacked', 'hack@hack.com', '000'); --"

// 实际执行的 SQL
INSERT INTO users (username, password, email, phone)
VALUES ('hacker', 'hacked', 'hack@hack.com', '000'); --, 'ignored', 'ignored', 'ignored')
↑ 攻击者闭合 VALUES 并执行了自定义 INSERT
```

结果: 攻击者成功插入自定义数据, 后续字段被 -- 注释忽略。

4.4 攻击危害总结

攻击类型	Payload 示例	后果
数据泄露	' OR '1'='1	获取全部用户信息
表结构探测	' UNION SELECT ... FROM sqlite_master--	获知完整数据库结构
数据窃取	' UNION SELECT id,username,password,phone FROM users--	窃取所有用户的密码哈希
数据篡改	注册注入 UPDATE/DELETE	修改或删除数据库记录

5. 修复方案

5.1 核心修复：使用参数化查询 (Prepared Statement)

SQL 注入的根治方法是将 **SQL** 代码与数据分离。参数化查询使用 ? 占位符，数据库驱动会将用户输入始终作为数据处理，永远不会解析为 SQL 命令。

5.2 搜索路由修复

```
// ❌ 修复前：f-string 拼接，存在注入
sql = f"SELECT ... WHERE username LIKE '{keyword}%' OR email LIKE '{keyword}%'"
c.execute(sql)

// ✅ 修复后：参数化查询，安全
sql = "SELECT ... WHERE username LIKE ? OR email LIKE ?"
c.execute(sql, (f"%{keyword}%", f"%{keyword}%"))
```

5.3 注册路由修复

```
// ❌ 修复前：f-string 拼接，存在注入
sql = f"INSERT INTO users (...) VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)

// ✅ 修复后：参数化查询，安全
sql = "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)"
c.execute(sql, (username, password, email, phone))
```


6. 修复前后代码对照

6.1 搜索路由 (app.py)

✗ 修复前

```
keyword = request.args.get("keyword", "")
sql = f"SELECT ... LIKE '%{keyword}%'"
c.execute(sql)
```

✓ 修复后

```
keyword = request.args.get("keyword", "")
sql = "SELECT ... LIKE ?"
c.execute(sql, (f"%{keyword}%",))
```

6.2 注册路由 (app.py)

✗ 修复前

```
username = request.form.get("username")
sql = f"INSERT INTO users VALUES\n('{username}', ...)"
c.execute(sql)
```

✓ 修复后

```
username = request.form.get("username")
sql = "INSERT INTO users VALUES (?, ?, ?, ?)"
c.execute(sql, (username, ...))
```

7. 修复验证

#	测试用例	结果
1	正常搜索 "admin" 返回正确结果	✓ 通过
2	搜索 ' OR '1'='1 注入 — 不返回额外数据	✓ 通过
3	搜索 UNION SELECT 注入 — 不泄露表结构	✓ 通过
4	正常注册新用户后搜索可见	✓ 通过
5	注册注入恶意 username — 作为纯文本存储，不执行 SQL	✓ 通过
6	SQL 日志仍可正常输出调试	✓ 通过

验证日志（服务器控制台）

```
# 正常搜索 — 参数化查询日志
[SQL] SELECT ... WHERE username LIKE ? OR email LIKE ? | 参数: keyword=admin
[SQL] 命中 1 条记录

# 注入尝试 ' OR '1'='1 — 参数被当作纯文本，不解析为 SQL
[SQL] SELECT ... WHERE username LIKE ? OR email LIKE ? | 参数: keyword=' OR '1'='1
[SQL] 命中 0 条记录 ← 注入失败，安全

# UNION 注入尝试 — 同样被参数化拦截
[SQL] SELECT ... WHERE username LIKE ? OR email LIKE ? | 参数: keyword=' UNION SELECT ...
[SQL] 命中 0 条记录 ← 注入失败，安全
```

8. 修复总结

#	漏洞名称	位置	原等级	状态
9	搜索功能 SQL 注入	/search	● 严重	✓ 已修复
10	注册功能 SQL 注入	/register	● 严重	✓ 已修复

改动文件

文件	变更
app.py (register 路由)	f-string INSERT → 参数化 INSERT (?, ?, ?, ?)
app.py (search 路由)	f-string SELECT LIKE → 参数化 SELECT LIKE ?

安全建议

- 始终坚持参数化查询：任何时候拼接 SQL 都应使用 ? 占位符，禁止使用 f-string 或 % 格式化
- 最小化数据库权限：应用账号应仅拥有所需的最小权限（如搜索只读，注册只写）
- 输入验证：对用户名、邮箱等字段做格式校验（如邮箱正则匹配），减少攻击面
- **SQL** 日志保护：生产环境日志不应包含完整的 SQL 语句或用户参数
- 定期代码审计：使用自动化工具（如 SQLMap、Bandit）扫描 SQL 注入风险

两轮审计汇总

轮次	审计内容	漏洞数	严重	中危	状态
第 1 轮	密码安全	8	4	4	✓ 已修复
第 2 轮	SQL 注入	2	2	0	✓ 已修复
合计		10	6	4	全部已修复

安全漏洞审计报告 · 第2轮 — SQL 注入安全 | 报告日期：2026-07-08

项目地址：<https://github.com/sencly-book/user-management>